

Making 6DOF Mods 3D

A General Method

Revision 4 — the stable AFR engine. Frame-sequential output removed; the method is rebuilt around alternate-frame (AFR) stereo capture composited so both eyes live in every presented frame, with the full no-swap / no-flicker / no-freeze stability stack proven across six shipped 6DOF mods and decoded from open VR mods (UEVR, cyberpunk-vr-port, Luke Ross R.E.A.L.).

What changed in Revision 4. The document no longer describes frame-sequential (shutter) output at all. Every stereo path here composites both eyes into the presented frame — spatial packings (SBS / TAB / line / column / checker) or the full-resolution VR shared surface. The old ‘eye clock’ is replaced by a **present-driven eye flip**, and an entire new engine section documents the accumulated AFR stability fixes: warmup gate, QPC timing, fps-adaptive stale window, align-to-fresh reprojection, high-resolution pacer, frame-latency cap, and the pause-menu idle guard. The geometry (parallax law, tiers, separation, HIT convergence, FOV/Z independence) is unchanged and carried forward verbatim where it was already correct.

Scope. A camera-hook 6DOF mod already moves the game’s own camera additively (look, lean, peek). This document adds *real geometric stereo* on top of that camera, as a present-time layer that never touches the tracking code. The two layers share one camera and simply add, so a mod can be tracking-only, 3D-only, or both.

Honest limit, stated once. Real stereo from a camera hook costs the second eye. AFR pays it by alternating which eye the engine renders each frame, so each eye refreshes at half the game’s frame rate while the *display* shows both eyes every frame. The stability stack below removes every artifact that is *wiring* (swap, flicker, freeze); the one artifact that is *physics* (one eye is one frame older than the other during motion) is minimized by reprojection, not pretended away.

Contents

#	Section
Part I	The geometry (unchanged)
1	What this method produces, and its honest limits
2	The one formula every geometric 3D tool uses
3	The parallax equation, derived
4	Camera-space realisation: separation offset + HIT convergence
5	Accurate 3D values and worked examples
6	Know your mod: the three camera-access tiers
7-9	Tier A / Tier B / Tier C realisations
10-11	Convergence as one distance; FOV and Z as independent controls
Part II	The stable AFR engine (the core of Rev 4)
12	AFR capture: the camera alternates, Present owns the eye
13	Present-driven eye flip - clean 1:1 alternation at any fps
14	Persistent slots and SwapEyes-applied-once
15	The AFR stability stack - every fix, in order
16	The cadence log - verifying it on the first run
Part III	Output: both eyes in every presented frame
17	Output modes 0-6 - the single compositor shader
18	The VR shared-surface contract (full resolution)
19	DX11 vs DX12 present-hook plumbing
Part IV	Integration
20-24	Independence, version-agnostic hooking, INI, checklist, pitfalls
App.	Decoded lineage: UEVR, cyberpunk-vr-port, Luke Ross; conventions; safety

Part I — The geometry (unchanged)

1. What this method produces, and its honest limits

The mod outputs **real geometric stereo**: two views of the scene from two eye positions offset along the camera's right axis, so parallax is correct at every distance — near objects diverge, the convergence plane sits on the screen, far objects recede. It is not a 2D-to-3D trick and not a fixed per-object shift. Because the offset rides on the head-tracked camera, you get **full 6DOF in stereo**: look, lean and peek, each eye rendered from the correct displaced viewpoint.

Both eyes are always **composited into the presented frame** — either a spatial packing for a 3D monitor or a full-resolution VR shared surface. There is no shutter-glasses / frame-sequential display path in this revision: that path is where the entire historical swap-and-flicker class lived, and it is unnecessary once the compositor holds both eyes.

Honest limits

- + **Second-eye cost.** Real geometry needs the scene drawn from two viewpoints. AFR gets the second viewpoint by alternating the camera each frame, so per-eye refresh is $\text{game-fps} / 2$. The display still shows both eyes every frame; motion introduces a one-frame inter-eye age difference that reprojection minimizes (section 15.5).
- + **View-dependent shading** (reflections, some post) is rendered per real eye, so it is correct — this is an advantage over depth-reprojection methods, which cannot.
- + **Temporal effects** (motion blur, TAA, film grain) sample across frames and therefore disagree between the two alternating eyes; they must be disabled in the game's own settings (section 15.9).

2. The one formula every geometric 3D tool uses

Clip-space stereo drivers, depth-reprojection shaders and this camera-offset method are algebraically the same operation. A stereo driver adds, in the vertex stage before the perspective divide:

```
clip.x += separation * (clip.w - convergence)    // per eye, eyeSign on separation
```

Dividing the clip-space shift by clip.w gives the on-screen (NDC) disparity a viewer sees for a point at view depth d:

```
NDC_disparity(d) = separation * (d - convergence) / d  
                  = separation * ( 1 - convergence / d )
```

That is the whole method. **Separation** is the eye offset (a driver's Divergence / strength); **convergence** is a depth — the distance where disparity is zero, i.e. where things sit on the screen (a driver's ZPD). Everything else in Part I is how you realise those two terms given what your mod can reach.

3. The parallax equation, derived

Depth d vs convergence C	NDC disparity	Where the object appears
$d > C$ (farther)	positive, $\rightarrow$ separation as d grows	recedes BEHIND the screen
$d = C$	zero	sits ON the screen (zero-parallax plane)
$d < C$ (nearer)	negative	pops OUT in front of the screen

Master law: $\text{disparity} = \text{separation} \times (1 - \text{convergence} / \text{depth})$. Convergence is a real depth; separation is a real distance (the eye offset). Expose both in physical units and the same numbers transfer across engines.

4. Camera-space realisation: separation offset + HIT convergence

Separation — a parallel camera offset (exact)

Offset each eye by $\pm\frac{1}{2}\cdot\text{IPD}$ along the camera's right axis, cameras kept parallel. This is geometrically exact, gives zero-parallax at infinity, and is reachable on every tier. In AFR it is applied **in the camera cave** (the mod nudges the tracked camera by the eye offset for whichever eye this frame belongs to).

```
// in the cave, after head-tracking has composed the final camera basis:
rightAxis = normalize(cameraBasis.right);
eyeOffset = eyeSign * separationUnits; // eyeSign = -1 left, +1 right
cameraPos += rightAxis * eyeOffset; // parallel offset, C = 0 in-camera
```

Convergence — a constant image shift (HIT)

Because the AFR camera offset is **parallel** (convergence $C = 0$ in the camera), the convergence plane is set later, by the compositor, as a Horizontal Image Translation: subtract a constant per-eye image shift $h = (t/2) \cdot P00/C$. The net disparity becomes:

```
disparity(d) = t * P00 * ( 1/d - 1/C ) // parallel camera offset + HIT in the packer
```

This is the key division of labour in AFR stereo: **separation lives in the camera, convergence lives in the shader** as the HIT. The shader term is exactly $\text{srcUV}.x - \text{eyeSign} \cdot g\text{Conv}$ (section 17).

Projection convergence — exact, if you can reach it (Tier C)

If the mod can edit the projection matrix (Tier C, DX11 cbuffers), the HIT can be applied exactly and keystone-free in the projection instead of the packer:

```
// row-major DX projection; P00 is x-scale, P[2][0] maps view-z into clip-x:
P[2][0] += eyeSign * (t/2) * P00 / C; // == the HIT / convergence, exactly
```

For AFR the packer HIT is preferred because it is engine-agnostic (needs no projection access) and, since the compositor already runs, free.

5. Accurate 3D values and worked examples

Separation is a real distance, so set it from a real IPD. Human IPD averages ~63–65 mm (range ~55–75). Per eye you offset half of it; convert to game units with WorldUnitsPerMetre (WUPM):

```
sepPerEye_units = (IPD_mm / 1000 / 2) * WUPM = IPD_mm * 0.0005 * WUPM
```

IPD (mm)	per-eye	@WUPM=1	@WUPM=10	@WUPM=100	@WUPM=150
55	27.5 mm	0.0275	0.275	2.75	4.13
65 (default)	32.5 mm	0.0325	0.325	3.25	4.88
75	37.5 mm	0.0375	0.375	3.75	5.63

Why WUPM matters. Engines in this project span three orders of magnitude: RAGE (GTA V) and AnvilNext (AC Syndicate) use 1 unit = 1 m (WUPM=1); Dawn (Guardians) ~10; UE3 (Batman AK) ~100–150. A per-eye separation is 0.0325 units on RAGE and 3.25–4.88 on UE3 — a 100× spread. Any value exposed in raw world units is meaningless across engines; expose IPD in mm and convert via WUPM.

Convergence C	in metres @WUPM=150	feel
150 units	1.0 m	strong pop-out, close plane
350 units (default)	2.3 m	balanced third-person
600 units	4.0 m	gentle, deep scene

Comfort. Keep the total on-screen disparity of the farthest objects within roughly the viewer's IPD. In AFR the practical knobs are SeparationUnits (strength) and AfrConvergence (the HIT); tune separation first, then convergence, both live (section 22).

6. Know your mod: the three camera-access tiers

Which stereo realisation you use depends only on what the mod can already write to move the camera. This is the same taxonomy as the head-tracking families.

Tier	What the mod writes	Separation	Convergence
A	camera position + orientation (Euler/quat)	offset position along right axis	HIT in packer (recommended)
B	the output view matrix (4x4)	offset the matrix translation row	HIT in packer
C	view / projection cbuffers (DX11)	offset view; or projection shift	exact P[2][0] shift, or HIT

Guardians (Dawn, DX12) is Tier B: a 4x4 row-major camera matrix at camera+0x20. **Batman AK** (UE3, DX11) is Tier A: packed-Euler yaw/pitch/roll + position. Both apply separation in the cave and convergence as the packer HIT, so the two mods share one compositor.

7-9. Tier realisations, briefly

Tier A - position + orientation

Rebuild the camera's right axis from the (head-tracked) orientation, then add $\text{eyeSign} \times \text{sepUnits}$ along it to the position. For a packed-Euler camera (Batman), derive the right vector from the final yaw the mod already computes.

Tier B - output view matrix

The right axis is a row (or column) of the basis already in the matrix. Add the eye offset to the translation row along that axis (Guardians: the right row of the 4x4 at +0x20, post head-track).

Tier C - view / projection cbuffers

Offset the view matrix as in Tier B, or — uniquely available here — apply convergence exactly in the projection with the $P[2][0]$ shift. Watch for a cached $\tan(\text{fov}/2)$ pair if you also touch FOV.

10-11. Convergence as one distance; FOV and Z as independent controls

Expose convergence as a **distance**, never a bare angle. In AFR it is realised as the packer HIT (AfrConvergence), so a single scalar moves the zero-parallax plane for both eyes symmetrically. FOV and a Z-axis offset are **independent layers**: FOV scale/offset is a separate control from tracking and from 3D; the Z-axis offset slides the camera along forward for *both* eyes (it is not stereo, it is framing). Keep each on its own switch so it can be tested with the other three off.

12. AFR capture: the camera alternates, Present owns the eye

AFR renders one eye per presented frame: on even frames the camera carries the left-eye offset, on odd frames the right. The compositor keeps the two most recent eye images and shows them together every frame, so the *display* is always stereo even though each eye is produced every other frame.

The single hardest problem in AFR is **which eye a given frame belongs to**, because the engine does not write the camera on a clean one-per-frame cadence: it writes it **zero** times during menus and cutscenes and **one to seven** times per frame during gameplay (Guardians: 4 job threads). If eye identity is inferred from present-count parity, it desynchronises — and every swap, flicker and phase-drift bug in the history of these mods traces to exactly that inference.

The rule

The camera publishes the eye it applied; Present reads it; Present never re-derives it. The camera cave, running on the game thread, is the only thing that knows which eye it just rendered. It latches that eye once per frame; the present hook stores the finished frame into the slot for *that* latched eye. This is the injected-mod equivalent of an engine's internal render-frame counter (which UEVR has and an injected hook does not), and it is what makes the whole engine stable.

13. Present-driven eye flip — clean 1:1 alternation at any fps

The eye must alternate exactly once per *displayed* frame. The robust way to guarantee that is to flip it **in the present hook** — the one event that is truly once-per-displayed-frame — not in the camera cave. The cave only *reads* the current eye and applies its offset; the present hook flips it after storing the frame.

```
// ---- camera cave (game thread): read only, never flips ----
int Stereo_CurrentEyeSign() { return g_frameEye.load(acquire) ? +1 : -1; }

// ---- present hook, once per present ----
eye = g_frameEye.load();           // the eye the cave rendered this frame with
store(bb -> slot[eye]);           // (fps-adaptive stability, section 15)
if (cameraRanThisFrame)           // a new frame was actually rendered
    g_frameEye.fetch_xor(1);       // flip ONCE for the next frame + count it
```

Why present-driven and not cave-driven. An earlier design flipped the eye on the first camera write of each frame, gated by a handshake flag. It fails on real games for two reasons the cadence log exposes as `eyeFlips/s < present/s`: the engine writes the camera an *irregular* number of times per frame (Guardians' four job threads, some writes failing the matrix-validity check), and external tools can **manipulate time** (Cheat Engine speed hacks) so the write cadence no longer tracks the present cadence. Any eye clock tied to camera writes then alternates irregularly — one eye updating twice while the other goes stale — which reads as flicker. Flipping in the present makes the alternation a clean 1:1 no matter how the game writes the camera or scales its clock. Because all of a frame's camera writes read the *same* `g_frameEye` (it only changes at present), every write in a frame still applies the same eye offset.

The one caveat: a deep GPU pipeline can present a frame rendered a few frames after the eye was read, so which physical eye maps to slot 0 can land on either parity — a *constant* left/right reversal, fixed once with `SwapEyes` (Ctrl+F12), never a per-frame swap. Clean alternation is the property that matters; a one-time calibration handles the rest.

14. Persistent slots and SwapEyes-applied-once

Two GPU textures are the eye slots: **slot 0 is LEFT forever, slot 1 is RIGHT forever**. The present stores the finished frame into `slot[g_frameEye]`. Nothing ever swaps the slots. If the physical left/right come out reversed for a given game (the offset sign or the engine's right-axis convention differs), a single `SwapEyes` flag flips the routing in the shader — applied once, to eye + HIT + reproject together — and is a one-time calibration, never a per-frame decision. Persistent slots + apply-once are why a dropped frame can only ever stale an eye, not exchange the two.

15. The AFR stability stack — every fix, in order

The following nine mechanisms are the accumulated result of auditing these mods and decoding three open VR mods. Each is traceable to a specific failure it removes. Together they are what make AFR read as stable, flicker-free, freeze-free 3D.

15.1 Persistent-slot handshake (sections 13–14)

The no-swap foundation. Everything else assumes it.

15.2 AER warmup gate — kills the Mono->AER stall and first-pair freeze

When stereo is first enabled (or the slots are (re)built), the pipeline has content for at most one eye. Compositing a pair now shows a half-filled pair — a freeze or flash. Gate it: on slot build, set a small warmup countdown and invalidate both slots; force *fresh-in-both* (mono) until **both** eyes have been captured, then settle a few frames. Decoded from cyberpunk-vr-port, whose pair id must be coupled to the eye-0 capture, not present-count parity, or ~50% of enables start mid-pair and never complete (blank HMD).

```
int warm = g_afrWarmup.load();
if (warm > 0) {
    if (g_slotValid[0] && g_slotValid[1]) g_afrWarmup.store(warm - 1); // settle once both eyes exist
    if (g_slotValid[fresh]) { forceMono = 1; freshIsLeft = (fresh==0); } // show the one eye we have
}
```

15.3 QPC-accurate eye timing

The inter-eye gap is ~11 ms at 90 fps — finer than GetTickCount64's ~15 ms granularity, which makes a millisecond-based stale test pure noise at high fps. Time everything with QueryPerformanceCounter. Every timestamp below (slot capture time, frame-period EMA) is QPC.

15.4 No time-based mono fallback — it was the flicker

An earlier design forced the whole image to mono whenever the two eyes drifted too far apart in *time* (a 'stale window'). Testing showed this is the wrong idea: it fires on every ordinary frametime hitch, so the picture collapses to mono and back on each spike — read on-screen as **flicker between the eyes**. Worse, it measures *time*, so an external clock scaler (Cheat Engine speed hacks) makes it fire constantly, which is exactly the 'the game flickers when I enable speed hack' report. The fix is to **remove it entirely**: during gameplay the compositor always shows the real pair (an eye that is a few frames old is a touch of doubling, far less objectionable than a mono blink). The only thing that should ever force mono is a genuine *camera stop* — and that is detected by **frame count**, not time (15.10), so it is immune to both hitches and time manipulation.

What remains in this branch is only the optional reprojection (15.6); the two eyes are shown as-is otherwise. AFRStaleMs is now informational only.

15.5 Timestamp-based fresh detection (no eye-0 bias)

Pick which eye is 'fresh' purely by newest capture timestamp, never by a pair counter. cyberpunk-vr-port found pair-id detection makes one eye raw more often (the counter only bumps on eye-0), producing a persistent 'right eye worse than left' asymmetry. Capture age is unbiased.

```
int fresh = (slotTimeMs[0] >= slotTimeMs[1]) ? 0 : 1; int stale = fresh ^ 1;
```

15.6 Align-stale-to-fresh reprojection — OFF by default, and why

The compositor *can* shift the stale eye horizontally by the measured inter-eye viewpoint-yaw delta, to align it to the fresh eye's instant. In principle this reduces the temporal mismatch during rotation. In practice, the shift is proportional to the per-frame yaw change, so under **variable frametime** (uncapped play, hitches) the shift jitters frame-to-frame and reads as **judder while panning** — especially with an analog stick, where yaw changes continuously. Two mitigations, in order of importance: (1) it now ships **off by default** (AfrReproject=0) — each eye then shows its true rendered content, one frame apart, which the eyes fuse cleanly; (2) when enabled, the shift is **low-pass filtered** so a noisy frametime can no longer make it jump:

```
float d = slotYaw[fresh] - slotYaw[stale]; // measured inter-eye viewpoint-yaw delta
unwrap(d);
float shift = d * AfrReprojGain * 0.98f; clamp(shift, +/- AfrReprojClamp);
reprojEMA += (shift - reprojEMA) * 0.25f; // low-pass:kill per-frame jitter
```

```
shift = reprojEMA; // smoothed shift, then fed to the shader per slot
```

15.7 High-resolution frame pacer

Even eye spacing needs even present spacing. Pace *before* the real Present (SpecialK style) with a high-resolution waitable timer that OS-waits the bulk of the interval and spins only the last ~0.5 ms — never a fixed short cap that falls through to a multi-millisecond busy-spin.

```
due = -(remaining - 0.0005) * 1e7;
SetWaitableTimer(timer, &due, ...);
WaitForSingleObject(timer, (DWORD)(remaining*1000.0) + 2); // OS waits the bulk (low CPU)
do { QueryPerformanceCounter(&now); } while (elapsed < target); // spin only the final ~0.5 ms
```

15.8 Frame-latency cap = 1

Fewer frames in flight = tighter eye pairing. Luke Ross R.E.A.L. limits D3D max frame latency to 1; do the same: DX11 IDXGIDevice1::SetMaximumFrameLatency(1), DX12 IDXGISwapChain2::SetMaximumFrameLatency(1) where supported.

15.9 Temporal-effect suppression (in-game settings)

Once swap and staleness are solved, the only remaining flicker is **per-eye temporal effects**: they sample across frames, so the two alternating eyes disagree. UEVR force-disables motion blur via Unreal CVars ('it freaks out with AFR'); an injected mod cannot reach those CVars on Dawn / UE3, so the required settings are documented for the player instead:

- + **Motion Blur** OFF (the single biggest AFR offender).
- + **Anti-aliasing**: prefer SMAA / MSAA / FXAA over TAA / TSR / DLSS / FSR (temporal upscalers alternate eyes).
- + **Film Grain, Chromatic Aberration, Depth of Field, Lens Flare** OFF.
- + **Auto-exposure / Eye Adaptation** minimal (per-eye brightness mismatch).

15.10 Pause-menu / cutscene idle guard — no freeze, and keep the format

During a pause menu, cutscene or photo mode the game **stops writing the camera but keeps presenting the menu into the back buffer**. Hold-idle then leaves the slots holding the last gameplay pair, and compositing that pair over the menu overwrites it — the screen looks frozen and the menu is invisible. The fix: count consecutive idle frames and, after a few, copy the **live** back buffer into a slot and composite it into **both eyes** — but **still in the current output format** (SBS / TAB / interlaced / VR), never a raw mono passthrough. A 3D display therefore keeps its packing while the menu shows (as mono content in that format), and the real pair re-warms on resume. Presenting the raw back buffer instead would blank a 3D display's format — which is why the guard composites rather than bails. The threshold is counted in **frames** (~20, about a fifth of a second), not milliseconds, so a transient job-thread skip during gameplay never trips it and a speed-hacked clock cannot either — only a sustained camera stop does.

```
if (wrote) { idleFrames = 0; }
else if (++idleFrames > kIdlePassthrough) { // ~20 FRAMES (not time): a real stop only
    copy(bb -> slot[0]); forceMono = 1; freshIsLeft = 1; // both eyes = live frame, no HIT
    composite(currentOutputMode); // KEEP the format (SBS/TAB/interlace/VR)
    on-resume: g_afwWarmup = 8; // re-warm the real pair
    /* eye still flips once per present (section 13) */
    return true; // presented the live frame IN the current format
}
if (wasIdle && wrote) { wasIdle = false; g_afwWarmup = 8; } // gameplay resumed -> re-warm
```

This is the AFR analogue of cyberpunk-vr-port's 'submit mono when a menu is active', but kept in-format so a 3D display never loses its packing. The composite still runs through the normal state save/restore, so the menu is never corrupted by the stereo pass.

16. The cadence log — verifying it on the first run

Because the stability is structural, one log line proves it on the first real run. Emit once per second:

```
[3D][cadence] present/s=.. camWrite/s=.. frametime min/max/jitter=.. eyeflips/s=.. held/s=..
```

- + **eyeflips/s ~ present/s** means clean 1:1 alternation (each frame flipped the eye exactly once).
- + **held/s low** means few frames had no camera write during gameplay (few stale eyes).
- + **frametime jitter low** means the pacer is holding even spacing.
- + **camWrite/s -> 0** is the signature of a menu/cutscene (the idle guard is compositing the live frame in the current format).

17. Output modes 0–6 — the single compositor shader

One full-screen pass composites the two persistent slots (t0 = LEFT, t1 = RIGHT) into the presented frame. It runs at Present, over the final display-ready image, so it is tier-independent and works identically on DX11 and DX12. Per *output* pixel it (1) selects the eye from the mode's packing rule, (2) maps to the source UV, (3) applies the convergence HIT and the stale-eye reprojection shift, (4) samples the correct slot — with a fresh-in-both fallback and SwapEyes folded into the same branch.

The constant buffer (12 floats)

```
cbuffer Params : register(b0) {
    float gConv;          // per-eye convergence HIT (uv) = ConvergenceScale * sepUnits / ConvDistance
    float gOutW, gOutH;    // output surface size (W,H for 1..5; 2W,H for mode 6)
    float gMode;           // 0..6
    float gSwapEyes;       // 1 -> flip eye routing (one-time calibration)
    float gShiftL, gShiftR; // stale-eye reprojection shift per slot (uv), set by the AFR engine
    float gForceMono;      // 1 -> both eyes read the FRESH slot (warmup / hitch / idle)
    float gFreshLeft;      // 1 -> the fresh slot is LEFT (t0), else RIGHT (t1)
    float3 _pad;
};
Texture2D texL:register(t0); Texture2D texR:register(t1); SamplerState sPoint:register(s0);
// POINT filter, MIRROR address: the convergence HIT shifts each eye off the seam; MIRROR
// reflects the revealed strip instead of smearing a clamped edge (the visible center seam).
```

POINT sampling is mandatory (Appendix C): a linear tap bleeds the other eye across the pack seam or the interlace boundary. The address mode is **MIRROR**, not clamp: the convergence HIT pushes each eye away from the seam, and where the sample leaves [0,1] a clamp would smear the edge pixel into a visible strip right at the center seam — MIRROR reflects the adjacent image there instead, so the seam (and the outer edges) read as a clean symmetric fold. The shader therefore does not saturate the shifted coordinate; it lets the mirror address handle it.

The vertex stage — a full-screen triangle, no vertex buffer

```
VSOut VSMain(uint id : SV_VertexID) {
    VSOut o; o.uv = float2((id<1)&2, id&2);
    o.pos = float4(o.uv*float2(2,-2)+float2(-1,1), 0, 1); return o;
}
```

Per-mode eye selection and source mapping

Every mode reduces to two things: a boolean `left` for this output pixel, and the source UV to read. Modes 1–2 *pack* (each half stretches to full), so the source UV is rescaled; modes 3–5 *interlace* (both eyes at full frame, the panel selects), so the source UV is the output UV.

#	Name	left =	source UV	Display / panel
0	Off / mono	-	uv	passes the LEFT slot through unchanged
1	SBS	uv.x < 0.5	(frac(uv.x*2), uv.y)	side-by-side 3D TV / player
2	TAB	uv.y < 0.5	(uv.x, frac(uv.y*2))	top-bottom 3D TV / player
3	Line	floor(y) even	uv	row-interlaced passive 3D monitor
4	Column	floor(x) even	uv	column-interlaced panel
5	Checker	(floor(x)+floor(y)) even	uv	checkerboard 3D DLP
6	VR full-res	uv.x < 0.5	(frac(uv.x*2), uv.y)	2W x H shared surface -> VR viewer

Modes 1–5 are half-resolution per eye **by format/panel law** — the packing or the panel splits the pixels — not by any implementation flaw. **Mode 6 is the only full-resolution-per-eye route**: the target is 2W×H, the left half is the entire left eye and the right half the entire right eye, so nothing is sub-sampled and both eyes coexist in one texture — making mode 6 additionally immune to the whole swap/phase class at the *surface* level (section 18). Mode 3–5 parity uses `SV_Position` (the integer pixel), never the UV, so it lands on exact panel rows/columns.

The pixel stage — selection, fallback, HIT, reproject, sample

```
float4 PSMain(VSOut i) : SV_Target {
```

```

int mode = (int)(gMode+0.5); float2 uv = i.uv; float px=i.pos.x, py=i.pos.y;
if (mode==0) return float4(texL.Sample(sPoint, uv).rgb, 1); // passthrough mono

float eyeSign; float2 srcUV; // -1 left, +1 right
if (mode==1 || mode==6) { bool L=uv.x<0.5; eyeSign=L?-1:1; srcUV=float2(frac(uv.x*2),uv.y); }
else if (mode==2) { bool L=uv.y<0.5; eyeSign=L?-1:1; srcUV=float2(uv.x,frac(uv.y*2)); }
else if (mode==3) { bool L=fmod(floor(py),2)<0.5; eyeSign=L?-1:1; srcUV=uv; }
else if (mode==4) { bool L=fmod(floor(px),2)<0.5; eyeSign=L?-1:1; srcUV=uv; }
else { bool L=fmod(floor(px)+floor(py),2)<0.5; eyeSign=L?-1:1; srcUV=uv; }

if (gForceMono>0.5) // warmup / hitch / idle: both eyes = the fresh slot
    return float4((gFreshLeft>0.5?texL:texR).Sample(sPoint, srcUV).rgb, 1);
if (gSwapEyes>0.5) eyeSign = -eyeSign; // one-time calibration: eye + HIT + reproject together
float extra = (eyeSign<0)?gShiftL:gShiftR; // stale-eye reprojection, matched to the sampled slot
float2 s = float2(saturate(srcUV.x - eyeSign*gConv + extra), srcUV.y); // convergence HIT
return float4((eyeSign<0)?texL:texR).Sample(sPoint, s).rgb, 1);
}

```

Why each term is correct. The HIT sign: left eye (eyeSign=-1) shifts +gConv, right shifts -gConv — the two eyes translate in opposite directions, which is exactly the convergence plane moving from infinity to a finite distance (the packer realisation of $\text{disparity}(d)=t \cdot P00 \cdot (1/d-1/C)$, section 4). The stale-shift is read from the same branch that selects the slot, so it always corrects the eye it is applied to. SwapEyes flips eyeSign before both the HIT and the sample, so routing, HIT direction and reprojection stay mutually consistent — a reversed-depth calibration can never desynchronise them. The fresh-in-both branch is taken *before* HIT/reproject, so a fallback frame is a clean mono image with no half-applied shift.

18. The VR shared-surface contract (full resolution)

Mode 6 publishes a 2W×H surface a VR viewer (Katanga / Osiris-class) opens by handle. The contract, decoded exactly:

- + The composite target is 2W×H; the left half is the full left eye, the right half the full right eye.
- + The surface is published via a 4-byte legacy handle written into a named file mapping (Local\KatangaMappedFile).
- + On **DX12** the NT handle is process-local and the viewer cannot open it, so the published surface must be owned by a private **D3D11** device on the same adapter; the D3D12 composite target is opened into D3D11 via a shared NT handle and copied under a shared fence.
- + On **DX11** the surface can be MISC_SHARED directly — no interop device needed.

Because both eyes live in one texture at fixed halves, mode 6 has no eye clock, no present phase and nothing to align at the surface level — the handshake still governs *capture*, but the *output* is inherently coherent.

19. DX11 vs DX12 present-hook plumbing

Both APIs hook DXGI Present at vtable slot 8, acquired by borrowing the vtable from a throwaway swapchain, then installed on the game's objects (the vtable is shared per class).

DX12

- + Also hook ID3D12CommandQueue::ExecuteCommandLists (vtable 10) to capture the game's DIRECT queue, so the composite is ordered after the game's own render.
- + Use a **3-allocator ring + fence-from-N-frames-ago**: a single allocator forces a full GPU drain every frame (up to 2× slower, worse when uncapped). Wait on the ring fence with a bounded timeout (1000 ms) — never an unbounded wait.
- + Copy the back buffer into the eye slot with resource-state barriers; format from the resource, never from the request.
- + Mode 6 needs the D3D11-interop surface (18).

The DX12 composite, in order (on an idle/menu frame the guard from 15.10 copies the live back buffer into slot 0 and composites it in-format instead of storing an eye):

```

bool wrote = g_camWrote.exchange(0); int eye = g_frameEye.load(); // published by the cave
if (wrote) idle=0; else if (++idle > 3) { g_frameOpen.store(1); return false; } // menu -> live bb
int r = ringIdx; // 3-allocator ring
if (ringFence[r] && fence->GetCompletedValue() < ringFence[r]) // wait on the fence from
    WaitForSingleObject(evt, 1000); // N frames ago, bounded
alloc[r]->Reset(); list->Reset(alloc[r], pso);
if (wrote) { // store THIS eye into its slot
    Barrier(bb, PRESENT->COPY_SOURCE); Barrier(slot[eye], PSR->COPY_DEST);
    list->CopyResource(slot[eye], bb); slotTimeMs[eye]=NowMs(); slotYaw[eye]=viewYaw;
    Barrier(slot[eye], COPY_DEST->PSR); Barrier(bb, COPY_SOURCE->RENDER_TARGET);
} else Barrier(bb, PRESENT->RENDER_TARGET);
// ... fresh/stale + warmup + reproject decide gForceMono/gShift* (section 15) ...
WriteParams(); SetDescriptorHeaps; SetGraphicsRootCBV; SetGraphicsRootDescriptorTable;
OMSetRenderTargets(bb); DrawInstanced(3,1,0,0); // full-screen triangle
Barrier(bb, RENDER_TARGET->PRESENT); list->Close();
queue->ExecuteCommandLists(1,&list); queue->Signal(fence, ++val); ringFence[r]=val;
ringIdx=(ringIdx+1)%3; g_frameOpen.store(1); // re-open the handshake

```

Format is read from the resource (bb->GetDesc()), never from the request; the intermediate colour copy matches it by construction, so no double-gamma or format mismatch (Appendix C). Mode 6 renders into the 2W×H bridge instead of bb, then signals the shared fence for the D3D11 copy (18).

DX11

- + Render the composite on the game's **immediate context**, wrapped in a full pipeline **state save/restore**: capture RTVs/SRVs/sampler/PSO/viewport/topology/blend before the draw and restore them after (an overlay that does not restore state corrupts the game's next frame).
- + No fence/ring is needed — the immediate context serialises — so there is no blocking wait at all on DX11.
- + IDXGIDevice1::SetMaximumFrameLatency(1) for tight pairing.
- + Mode 6 can use a MISC_SHARED surface directly — no interop device.

Fail-safe, both APIs: any failed HRESULT or bad pointer -> present the untouched mono frame. A stereo problem degrades to plain mono, never to a crash. On a menu the guard composites the live frame in-format through the same state save/restore, so the menu is shown correctly and never corrupted; and a single-instance mutex prevents two injected copies from fighting over the Present hook.

20. Independence — four layers, four switches (standard)

A 3D+6DOF mod is **four independent layers**, each on its own enable and its own hotkeys. Any combination is valid — tracking-only, 3D-only, FOV-only, a dolly with everything else off. No layer reads another's state; the stereo/FOV/Z layers never touch [Rotation]/[Position]/[Network], and the tracking layer never touches the others. Test each with the other three off.

Layer	INI section	What it does	Applied
6DOF tracking	[Rotation]/[Sensitivity], [Position]	head look / lean / peek on the shared camera in the cave, only when tracking is on	
3D stereo	[Stereo3D]	+/- half-IPD eye offset + compositor	in the cave + Present, only when Enabled=1
FOV	[FOV]	scales / offsets the game field of view	in the cave, whenever [FOV] Enabled=1
Z-dolly	[ZAxis]	slides the camera along forward, both eyes	in the cave, whenever [ZAxis] Enabled=1

Implementation rule. The cave must not early-return when tracking is off: the tracking toggle gates only the rotation/lean writes; FOV, Z and stereo run on their own switches. When tracking is off, the smoothed head values are zeroed so no stale head pose leaks into the stereo/Z basis, and the AFR eye-clock signal (Stereo_NoteCameraWrite) fires *before* any gate so 3D works with tracking disabled.

Standard hotkeys (ship these in every mod)

Keys	Action	Layer	Step (per tap)	Saved
Ctrl + F3 / F4	separation weaker / stronger	3D	3 mm of IPD	on release
Ctrl + F5 / F6	convergence nearer / farther	3D	3 distance units	on release
Ctrl + F8 / F9	FOV wider / narrower	FOV	0.05 scale	on release
Ctrl + Down / Up	camera closer / farther-back (dolly)	Z	0.10 m	on release
Ctrl + F11	cycle output mode 0..6	3D	-	yes
Ctrl + F12	swap eyes	3D	-	yes
End / Home	toggle tracking / recenter	tracking	-	-

Units are physical and engine-independent. Separation is mm of IPD and the dolly is metres, both converted to world units via WorldUnitsPerMetre at the point of use, so the same step feels the same on RAGE, Dawn and UE3. FOV is a scale multiplier. The sep/convergence steps are ~3 (a clear but not drastic tap); FOV and Z steps are set a touch larger in perceived effect (0.05 scale, 0.10 m) so a single tap is obvious without over-shooting.

Finding the FOV field. FOV is a projection parameter, not part of the view matrix, so its struct offset differs per engine and is not always adjacent to the camera. When it is unknown, ship the FOV layer Enabled=0 and log every nearby float in a plausible FOV range (radians ~0.3-2.0 or degrees ~20-140) once per second; zoom in-game and set [FOV] StructOffset to whichever offset changes, then enable it.

21. Version-agnostic hooking — and a stable camera cave

Find the camera write-site by AOB / struct-offset scan, never a fixed RVA, so a game patch does not break the mod. Hook Present by vtable slot (stable across versions). Provide an INI override for the offsets if an auto-detect ever misses. Retry the scan on a timer until gameplay is up.

Read the camera AFTER the write completes

The camera write is usually a multi-instruction copy (e.g. a run of movups [cam+0x20]/[+0x30]/[+0x40]/[+0x50] for a 4x4). Steal the **whole** copy into the code-cave and run those stolen bytes *first*, then read + additively modify the camera. Hooking at the *start* of the copy and reading mid-write yields a half-updated matrix (new rows mixed with old) — a classic source of camera jitter. Injectable camera tools (IGCS) intercept this exact instruction for the same reason; a head-tracking mod uses the identical site but *adds* to the result instead of replacing it.

Lock onto the main camera

The same copy instruction can also run for auxiliary passes — shadow maps, planar reflections, cubemaps — each with its own camera at a different address. Applying head-tracking or, worse, the per-eye stereo offset to those makes shadows and reflections shift per eye and **shimmer**. Lock onto the one main-view camera: the main view is written every frame so its address dominates. Strengthen a lock counter on the primary address and weaken it on any other; while the lock is strong, skip other addresses entirely (apply nothing to them). It re-locks within a few frames if the game legitimately switches camera (a cutscene). On a single-camera title this is a no-op; on a multi-camera one it removes a whole class of per-eye shimmer. A diagnostic that counts *auxiliary skips per second* tells you at a glance whether a given game is multi-camera.

Note on the AFR clock vs the lock. Keep the 'camera ran this frame' signal (which drives the present-driven eye flip and the idle guard) on *any* camera write, but apply the offset only to the locked primary. That way a menu — where *all* cameras stop — is still detected, while gameplay auxiliary passes never perturb the eye clock.

22. Standard file names

Use these across every mod so a user learns the layout once. The plugin and package carry the game name; the config and log are identical everywhere.

Artifact	Standard name	Example
Package (zip)	<Game>-3D-6DOF.zip	GuardiansOfTheGalaxy-3D-6DOF.zip
Plugin (asi)	<Game>-3D-6DOF.asi	BatmanArkhamKnight-3D-6DOF.asi
Config (ini)	3D-6DOF config.ini	(shared — identical layout everywhere)
Log	3D-6DOF.log	(shared)
Loader	dinput8.dll	Ultimate ASI Loader

The one legitimate exception to the plugin name: some games load through a statically-imported proxy DLL rather than an ASI loader (the Assassin's Creed titles import `version.dll`), so the mod *is* that DLL and forwards the real exports. There is no separate .asi to rename — renaming it would stop it loading. Keep the proven load path and apply the standard to everything that does not affect loading: config, log and package.

23. Full INI template — every option, organised, with hotkeys

Ship this exact layout in every mod. The output mode is the first choice; the four values a user actually tunes sit at the top of [Stereo3D] with a one-line note each; AFR stability and advanced keys follow. Tracking sections ([Network], [Rotation]/[Sensitivity], [Position], [FOV]) are the head-tracking mod's and are carried verbatim; the stereo layer never edits them.

```
[Stereo3D]
Enabled=1                ; 3D ON by default (standard). 0 = tracking-only (no stereo code runs)
; OutputMode: 0 OFF/mono | 1 SBS | 2 TAB | 3 Line | 4 Column | 5 Checker | 6 VR full-res (2WxH)
; 1-5 are HALF res per eye by FORMAT/PANEL law; only 6 is full-res. Two persistent slots +
; a published eye label => a dropped/repeated frame can STALE one eye but NEVER swap the eyes.
OutputMode=1             ; DEFAULT SBS (side-by-side)

; --- the four you tune (each a real physical value) ---
SeparationMM=65.0        ; 3D strength = IPD in mm (55-75). -> units via WorldUnitsPerMetre
ConvergenceDistance=350.0 ; distance (units) where things sit ON screen; nearer = more pop-out; 0=flat
SwapEyes=0               ; 1 if depth looks inside-out (applied once at slot selection)

; --- AFR stability (defaults are fine) ---
AfrStaleMs=40            ; floor for the stale window (ms)
AfrStaleFactor=1.5        ; real stale window = max(AfrStaleMs, this * smoothed frame period)
AfrReproject=1           ; align the stale eye to the fresh eye by measured viewpoint-yaw
AfrReprojectGain=0.011    ; uv shift per degree of yaw (~1/hFOV)
AfrReprojectClamp=0.06    ; hard cap on the reprojection shift (uv)
PresentSyncEyes=1        ; the eye is chosen ONCE per displayed frame (installs the handshake)
MaxFrameLatency=1        ; frames-in-flight cap (tighter eye pairing; Luke Ross R.E.A.L. = 1)

; --- performance ---
PresentUncap=1           ; 0 off | 1 VR-only (free) | 2 all modes (tearing on 1..5)
MaxFPS=0                 ; 0 = unlimited; else the pacer caps to a rate you can sustain
```

```

; --- advanced (leave unless calibrating) ---
ConvergenceScale=1.0          ; folds projection P00/FOV into the packer HIT (raise if convergence weak)
SeparationX=0.0              ; raw units/eye, used ONLY if SeparationMM=0
Convergence=0.0              ; direct uv HIT, used ONLY if ConvergenceDistance=0

; --- live tuning (edge-triggered: one step per tap, save on release) ---
SeparationStep=3.0           ; *** mm of IPD per TAP *** (same unit as SeparationMM)
ConvergenceStep=3.0          ; distance units per TAP (same unit as ConvergenceDistance)
AdjustModifier=0x11          ; held modifier for live keys (0x11=Ctrl, 0=none)
SeparationDownKey=0x72       ; F3      SeparationUpKey=0x73      ; F4
ConvergenceDownKey=0x74      ; F5      ConvergenceUpKey=0x75      ; F6
ModeCycleKey=0x7A           ; F11 cycle OutputMode
SwapEyesKey=0x7B            ; F12 swap eyes

[FOV]                        ; INDEPENDENT layer - works with tracking/3D on or off
Enabled=1                    ; FOV layer ON by default (Scale=1.0 = no-op until adjusted)
Scale=1.0                    ; multiplies game FOV (1.4 = 40% wider)
OffsetDegrees=0.0            ; adds a fixed amount (degrees)
StructOffset=0x60            ; FOV field offset in the camera struct (0 = use auto-detected)
FovStep=0.05                 ; scale change per TAP (noticeable, not drastic)
FovUpKey=0x77                ; F8 wider      FovDownKey=0x78      ; F9 narrower

[ZAxis]                      ; INDEPENDENT camera dolly - both eyes; works with tracking/3D off
Enabled=1                    ; metres along forward (-> units via WorldUnitsPerMetre)
Offset=0.0                   ; metres per TAP (noticeable dolly)
Step=0.15                    ; metres per TAP (noticeable dolly)
NearKey=0x26                 ; Up arrow (closer)   FarKey=0x28      ; Down arrow (farther)

[Position]
WorldUnitsPerMetre=150       ; MAIN calibration: metres -> game units. Scales SeparationMM,
                             ; ZAxisOffset and the mm<->units step conversion. (RAGE 1, Dawn 10, UE3 150)
; ... plus the head-tracking lean keys (Sensitivity/Invert/Limit/Row/Sign) ...

[Hook]
SiteRva=0                    ; 0 = auto (AOB scan + fallback); hex RVA override for other builds

[Hotkeys]
ToggleKey=0x23               ; End      toggle tracking      RecenterKey=0x24      ; Home      recenter

```

Live-tuning mechanics (three rules that all shipped as bugs first)

- + **The step is in the same unit as the value it steps.** SeparationStep is *mm of IPD* (matches SeparationMM); ConvergenceStep is *distance units* (matches ConvergenceDistance). A step in raw world units would mean 100× different things across engines (section 5) — on RAGE a 0.1-unit step is 300% of the whole separation.
- + **Convert at the point of use, exactly as the primary control does:** $\text{mm} = \text{clamp}(\text{cur_mm} \pm \text{step}, 0, 120)$; $\text{sepPerEye} = \text{mm} * 0.0005 * \text{WUPM}$.
- + **Keys must be edge-triggered.** A ~15 ms poll with a level test fires ~66 steps/second held, so a quick tap applies 5–10 steps. Fire once on the press edge, then auto-repeat only after a hold delay.

```

struct KRep { bool prev; DWORD t0, last; };
bool Rep(KRep& k, bool down) {
    const DWORD DELAY=400, RATE=90; // ms: hold 0.4s to repeat, then ~11/s
    DWORD now=GetTickCount(); bool fire=false;
    if (down && !k.prev) { fire=true; k.t0=k.last=now; }
    else if (down && now-k.t0>=DELAY && now-k.last>=RATE) { fire=true; k.last=now; }
    k.prev=down; return fire;
}

```

Every live change is written back to the INI so it persists across launches. On the release of an adjust key the poller writes the current values to disk; a mode-cycle or eye-swap saves on the tap. The keys and the fields they persist: separation → [Stereo3D] SeparationMM, convergence → ConvergenceDistance, mode → OutputMode, swap → SwapEyes, FOV → [FOV] Scale, dolly → [ZAxis] Offset. Each writer touches only its own layer's section — never a tracking key — and runtime values are seeded from the INI at launch, so the loop is closed: edit in-game, and the file already holds your setting next time.

Keys	Action	Step unit	Saved?
Ctrl+F3 / F4	Separation weaker / stronger	3 mm of IPD	yes, on release
Ctrl+F5 / F6	Convergence nearer / farther	3 distance units	yes, on release
Ctrl+F8 / F9	FOV wider / narrower	0.05 scale	yes, on release

Ctrl+Up / Down	Camera closer / farther (dolly)	0.15 m	yes, on release
Ctrl+F11	Cycle output mode 0..6	-	yes
Ctrl+F12	Swap eyes	-	yes
End / Home	Toggle tracking / recenter	-	-

24. Step-by-step porting checklist

- + Identify the tier (A/B/C) from what the mod writes to move the camera.
- + Measure WorldUnitsPerMetre first — every separation number depends on it.
- + Add the [Stereo3D] block; seed runtime values from it.
- + Apply **separation** in the cave along the view-right axis, gated by the eye handshake.
- + Install the Present hook (+ ECL on DX12); build the ring/fence (DX12) or state save (DX11).
- + Wire the compositor and the **convergence HIT**; confirm modes 0–6.
- + Enable the stability stack: warmup, QPC timing, adaptive stale window, reproject, pacer, latency cap, idle guard.
- + Ship the standard defaults (Enabled=1, OutputMode=1 SBS, [FOV] Enabled=1); set Enabled=0 for a tracking-only build. Bring up with the cadence log; tune separation, then convergence; SwapEyes if inverted; disable temporal effects in-game.

25. Common pitfalls and their fixes

Symptom	Cause	Fix
Eyes swap on a hitch	identity inferred from present parity	the frame-boundary handshake (13)
Flash/freeze when enabling 3D	pairing before both eyes exist	warmup gate (15.2)
Frozen screen on pause menu	stale pair drawn over the menu	idle passthrough (15.10)
Flicker between eyes	time-based mono fallback firing on hitches	respect handshake, show the real pair (15.4)
Game flickers under speed hack	time-measured stale/mono logic	frame-counted guards only (15.4, 15.10)
Constant flicker in motion	temporal effects across eyes	disable motion blur/TAA (15.9)
Depth inside-out	right-axis / offset sign	SwapEyes once (14)
Doubling on fast pans	inter-eye staleness	optional reprojection (15.6); it is off by default
Stutter / uneven 3D	uneven present spacing	high-res pacer + latency=1 (15.7-15.8)
Right eye worse than left	eye-0-biased fresh detection	timestamp fresh detection (15.5)

Appendices

A. Decoded lineage — where each fix comes from

Every mechanism in section 15 is traceable to a shipping mod, decoded from source where the mod is open.

UEVR (praydog, open source)

Eye = engine render-frame parity ($m_frame_count \% 2 == m_left_eye_interval$) using the engine's *internal render counter* — the ground-truth per-frame index an injected hook lacks, which is why we use the handshake as the equivalent. Its phase knob = our SwapEyes. Its Synced Sequential (*is_same_frame*: render the same frozen frame twice) needs engine-render control and is unreachable by injection, so AFR + reproject is the ceiling. Its force-disable of motion blur is our 15.9.

cyberpunk-vr-port (dariulone, MIT, open source)

Contributed, decoded from `openxr_present.cpp / openxr_frameLoop.cpp`: use the RENDERED eye not the scheduled one (confirms 13); the AER warmup (15.2); the first-pair-parity-freeze fix (pair id keyed to eye-0 capture); the 1.5-period fresh window (15.4); timestamp fresh detection removing the eye-0 bias (15.5); the -0.02 overshoot bias (15.6); submit-mono-on-menu (15.10). Its DBSR (depth reprojection) and NvOF optical-flow AER V2 are the higher-fidelity, hardware-heavy alternatives we deliberately do not use in an injected drop-in.

Luke Ross R.E.A.L. (published writeup; mod closed)

The AER pattern (100L·99R -> 100L·101R -> 102L·101R) matches persistent slots; reliance on the runtime's ATW for head-rotation doubling; and 'limited D3D11 max frame latency to 1' (15.8).

IGCS injectable camera (Otis_Inf; decoded from a shipping build)

The Guardians camera tool exposes a single `AOB_CAMERA_ADDRESS_INTERCEPT` — exactly the per-frame 4x4 matrix copy (`movups [cam+0x20..0x50]`) — confirming both the hook site and the +0x20 matrix layout used here; the trailing `movss [cam+0x60]` is a plain float in the copy, *not* the FOV. FOV is a separate site reached by its own `Fov` AOBs (a `mulss scale/clamp`), which is why writing +0x60 does nothing — find the real FOV field with the candidate-scan (section 11 note) or a dedicated FOV AOB. Its stability comes from intercepting that one instruction and controlling a single camera; the head-tracking mod adopts the same site and the main-camera lock (21), differing only in adding to the matrix rather than replacing it, and in never blocking the game's write.

B. Matrix, angle and coordinate conventions

Right axis is a basis row/column of the camera matrix (row-major DX: rows are basis vectors). Offset along the *normalised* right axis. If FOV is stored as a pair (raw radians + cached $\tan(fov/2)$), any FOV write must update both or it silently does nothing. Yaw sign and up/forward rows are engine conventions — expose them as INI so a mirrored axis is a config flip, not a rebuild.

C. Correct output composition

- + **POINT sampling, MIRROR address** for the compositor: never linear (a linear tap bleeds the other eye at the pack seam / interlace boundary), and mirror-not-clamp so the HIT/reproject-revealed edge at the seam reflects the image instead of smearing a clamped pixel.
- + **No double gamma**: the compositor writes display-ready pixels; do not re-encode.
- + Format from the resource, not the request; match the back-buffer format by construction.
- + Fail-safe passthrough on any error; single-instance mutex so two injected copies never fight over the Present hook.

D. Threading and safety

The cave (game thread[s]) and the present hook (present thread) share only atomics — no cross-thread lock, so no deadlock. The only bounded waits are the DX12 ring fence (1000 ms) and the pacer timer; the only loop is a ≤ 1 -iteration yaw unwrap. A single-instance mutex prevents a double-load. The stereo layer ships on by default now (Enabled=1), but is a single switch: set [Stereo3D] Enabled=0 and no graphics code runs — the mod is then exactly as safe as tracking-only.

E. The meta-lesson

Across every AFR defect ever seen in these mods, the parallax mathematics was never wrong — verified to 1e-16 — and every real failure was in the **wiring**: an eye identity inferred from the wrong clock, a gate that handed over on idle frames, a pair shown before both eyes existed, a composite drawn over a menu, an allocator nobody counted, an instrument raced by four threads. Revision 4's entire contribution is to make eye identity **intrinsic** (the camera publishes it, Present reports it) and to hold both eyes in the presented frame — so the swap, flicker and freeze classes cannot exist, and what remains is honest physics, minimized. Verify the arithmetic, then check it is plugged in — and check the instrument first.

Making 6DOF Mods 3D — A General Method, Revision 4 (stable AFR). Frame-sequential output removed. AFR capture + always-composited output, with the full no-swap / no-flicker / no-freeze stability stack. Companion to the 6DOF Head-Tracking Master Reference.